

# Medición de Tiempos de Interrupción

Sistema de Control de Acceso — FRDM-K64F

Santiago Riverós

Ignacio Sammartino

Rodrigo Devesa

Leandro Yiu

TP1 — Sistemas Embebidos

## Resumen

Este informe caracteriza experimentalmente los tiempos de interrupción del firmware mediante un pin de instrumentación medido con osciloscopio. El objetivo es verificar que el sistema cumple sus restricciones de tiempo real, cuantificar el costo de las abstracciones de la arquitectura en capas, y validar la asignación de prioridades de interrupción entre el SysTick y el lector de tarjetas.

## Índice

## 1. Método

El firmware incluye un módulo de instrumentación (`test_isr_cfg.h`) que pone un pin dedicado en alto al entrar a una rutina y en bajo al salir. En el osciloscopio, el **ancho del pulso** es el tiempo de ejecución de la rutina y el **período entre flancos de subida** es cada cuánto se ejecuta. La instrumentación se resuelve en tiempo de compilación: sólo la rutina seleccionada genera código, así que medir una rutina no afecta el tiempo de las demás.

| Parámetro              | Valor                      |
|------------------------|----------------------------|
| Microcontrolador       | MK64FN1M0VLL12 (Cortex-M4) |
| Frecuencia de core     | 100 MHz (1 ciclo = 10 ns)  |
| Optimización           | -O0 (build de Debug)       |
| Frecuencia del SysTick | 1 kHz (período 1 ms)       |
| Pin de medición        | PTC17                      |
| Osciloscopio           | _____                      |

Cuadro 1: Condiciones bajo las que se realizaron todas las mediciones.

Todos los tiempos corresponden a una compilación sin optimizar. Con -O2 los valores absolutos serían menores; las relaciones entre ellos, que sostienen las conclusiones de este informe, se mantienen.

## 2. Mediciones realizadas

Se realizaron nueve mediciones sobre las distintas fuentes de interrupción y tareas periódicas del sistema. Cada una activa una única rutina de instrumentación mediante la constante `TEST_ISR_TARGET`.

### 2.1. M1 — SysTick completo

**Configuración:** `TEST_ISR_TARGET = TEST_ISR_SYSTICK_TOTAL`

Mide la ejecución completa de `SysTick_Handler()`, la interrupción periódica de mayor jerarquía del sistema: despacha las tres tareas registradas (refresco del display, actualización del encoder y base de tiempo en milisegundos). Tiene que completarse siempre antes de que llegue el próximo tick, así que es la referencia contra la que se comparan las demás mediciones periódicas.

### 2.2. M2 — Refresco del display

**Configuración:** `TEST_ISR_TARGET = TEST_ISR_DISPLAY_REFRESH`

Mide `HAL_Refresh_Task()`, la tarea que se espera dominante dentro del SysTick: hace *bit-banging* de dos tramas de 16 bits (una de *blanking* anti-ghosting y una útil) hacia los shift-registers 74HC595. Se mide con el display a brillo máximo y a brillo mínimo, ya que el driver está escrito sin ramas por modo de operación y se espera que el ancho no cambie entre ambas condiciones.

### 2.3. M3 — FSM del encoder

**Configuración:** `TEST_ISR_TARGET = TEST_ISR_ENCODER_UPDATE`

Mide `actualizarEncoder()`, que lee tres pines y avanza una máquina de estados de antirrebote. El encoder se sondea por *polling* desde el mismo tick de 1 ms en lugar de usar interrupciones de pin propias; esta medición permite evaluar el costo real de esa decisión.

### 2.4. M4 — Base de tiempo en milisegundos

**Configuración:** `TEST_ISR_TARGET = TEST_ISR_TIMER_TICK`

Mide `Timer_Tick_Callback()`, la rutina más simple del sistema (un incremento de contador). Sirve como referencia del costo mínimo de un callback registrado en el *scheduler*, incluyendo la llamada indirecta a través del puntero a función.

### 2.5. M5 — Costo del despachador del SysTick

No requiere una medición directa ni una captura propia: se obtiene combinando los resultados de M1 a M4. `SysTick_Handler()` no ejecuta las tres tareas en línea, sino que recorre un arreglo de punteros a función y las invoca; ese bucle de despacho es el precio de que los módulos puedan registrarse en el *scheduler* sin que el MCAL los conozca. Su costo se calcula en la Sección ??.

### 2.6. M6 — Lector de tarjetas: captura de un bit

**Configuración:** `TEST_ISR_TARGET = TEST_ISR_CARD_CLK`, y luego `TEST_ISR_PORT_DISPATCH`.

Mide la ISR más crítica del sistema: `IRQ_CardClk()` se ejecuta una vez por cada bit leído de la banda magnética, y perder una sola ejecución corrompe la trama completa. Midiendo primero sólo el cuerpo de la ISR (`CARD_CLK`) y luego la interrupción completa con el despachador de puerto incluido (`PORT_DISPATCH`), se aísla el costo del mecanismo genérico de `gpioIRQDispatch()`, que recorre los 32 pines del puerto buscando cuál generó la interrupción.

## 2.7. M7 — Lector de tarjetas: apertura y cierre del *swipe*

**Configuración:** `TEST_ISR_TARGET = TEST_ISR_CARD_ENABLE`

Mide `IRQ_CardEnable()`, que se ejecuta dos veces por pasada de tarjeta (flanco de bajada al empezar, de subida al terminar). A diferencia de `IRQ_CardClk()`, esta ISR llama a `gpioIRQ()` para armar y desarmar la interrupción de `CLOCK`, lo que implica configurar registros del periférico (`PCR`, `ISFR`, `NVIC`); se espera un tiempo de ejecución notablemente mayor.

## 2.8. M8 — Latencia de interrupción

**Configuración:** `TEST_ISR_TARGET = TEST_ISR_CARD_CLK`, dos canales: `CH1` en `PTB10` (`CLOCK` del lector, la señal física) y `CH2` en `PTC17` (la respuesta del software).

Mide el tiempo entre el flanco de bajada de `CLOCK` y el instante en que el CPU efectivamente entra a la ISR. Esta medición permite validar experimentalmente la asignación de prioridades del `NVIC`: en `App_Init()` se le baja la prioridad al `SysTick` (`APP_NVIC_PRIO_SYSTICK = 1`) para que la interrupción de `PORTB` (prioridad 0, el lector) siempre pueda interrumpirlo. Si esa política funciona, la latencia medida debería ser pequeña y no acercarse nunca a la duración de la ISR del `SysTick` (M1).

## 2.9. M9 — Super-loop

**Configuración:** `TEST_ISR_TARGET = TEST_ISR_APP_LOOP`

Mide el período de `App_Run()`. No es una interrupción, pero determina la capacidad de respuesta de la aplicación: `encoder.h` documenta que un evento del encoder vive como máximo 1 ms antes de limpiarse, por lo que el super-loop debe completar una vuelta en menos de ese tiempo o se pierden pulsaciones. Se mide en reposo y durante la decodificación de una tarjeta, que es la carga adicional más pesada que puede caerle al lazo principal.

### 3. Resultados

#### 3.1. Anchos de pulso y períodos

| Ensayo | Rutina / condición              | Ancho         | Período          | Ciclos |
|--------|---------------------------------|---------------|------------------|--------|
| M1     | SysTick_Handler                 | _____ $\mu s$ | _____ ms         | _____  |
| M2     | HAL_Refresh_Task (brillo máx.)  | _____ $\mu s$ | 1 ms             | _____  |
| M2     | HAL_Refresh_Task (brillo mín.)  | _____ $\mu s$ | 1 ms             | _____  |
| M3     | actualizarEncoder (reposo)      | _____ $\mu s$ | 1 ms             | _____  |
| M3     | actualizarEncoder (girando)     | _____ $\mu s$ | 1 ms             | _____  |
| M4     | Timer_Tick_Callback             | _____ $\mu s$ | 1 ms             | _____  |
| M6     | IRQ_CardClk (solo la ISR)       | _____ $\mu s$ | _____ $\mu s$    | _____  |
| M6     | gpioIRQDispatch (con despacho)  | _____ $\mu s$ | _____ $\mu s$    | _____  |
| M7     | IRQ_CardEnable (apertura)       | _____ $\mu s$ | — (2 por pasada) | _____  |
| M7     | IRQ_CardEnable (cierre)         | _____ $\mu s$ | — (2 por pasada) | _____  |
| M9     | App_Run (reposo)                | _____ $\mu s$ | _____ $\mu s$    | _____  |
| M9     | App_Run (decodificando tarjeta) | _____ $\mu s$ | _____ $\mu s$    | _____  |

Cuadro 2: Resultados de todas las mediciones de ancho de pulso y período.

#### 3.2. Latencia de interrupción

La medición M8 es de naturaleza distinta a las anteriores: no es un ancho de pulso sino un retardo entre dos canales, así que se reporta aparte.

| Magnitud                            | Valor         | Ciclos |
|-------------------------------------|---------------|--------|
| Latencia mínima observada (CH1→CH2) | _____ $\mu s$ | _____  |
| Latencia máxima observada (CH1→CH2) | _____ $\mu s$ | _____  |

Cuadro 3: M8: latencia de interrupción del lector de tarjetas, sobre una serie de capturas.

## 4. Análisis

### 4.1. Cumplimiento de la restricción de tiempo real (M1)

Según la Tabla ??, el período de `SysTick_Handler` fue de \_\_\_\_\_ ms, lo que confirma que el SysTick está configurado correctamente a 1 kHz. La ISR consume \_\_\_\_\_  $\mu$ s de cada milisegundo, es decir un \_\_\_\_\_ % de la carga de CPU en ese contexto, dejando un \_\_\_\_\_ % de margen antes de comprometer el próximo tick. Ese margen equivale a \_\_\_\_\_  $\mu$ s: la ISR podría crecer esa cantidad de tiempo antes de que el sistema empiece a perder ticks.

### 4.2. Componente dominante del SysTick (M2)

El refresco del display consume \_\_\_\_\_  $\mu$ s, un \_\_\_\_\_ % del tiempo total de la ISR medido en M1: es, en efecto, el componente dominante. La diferencia entre brillo máximo y mínimo fue de \_\_\_\_\_  $\mu$ s. Dado que el atenuador está implementado por reparto de turnos de multiplexado — en los turnos “vacíos” se manda la misma trama con los segmentos apagados, en lugar de ejecutar menos código —, este resultado \_\_\_\_\_ (confirma / no confirma) que el costo de CPU del refresco es independiente del brillo elegido por el usuario.

### 4.3. Costo del *polling* del encoder (M3)

La FSM del encoder consume \_\_\_\_\_  $\mu$ s por tick, un \_\_\_\_\_ % del presupuesto de la ISR del SysTick. Es un costo \_\_\_\_\_ (despreciable / significativo) frente al del display, lo que respalda sondear el encoder por *polling* en el tick de 1 ms en lugar de dedicarle una interrupción de pin propia, que competiría por el NVIC con el lector de tarjetas.

### 4.4. Piso de costo de un callback (M4)

El contador de milisegundos, la rutina más simple del sistema, consume \_\_\_\_\_  $\mu$ s. Este valor representa aproximadamente el costo mínimo de cualquier callback registrado en el *scheduler*, incluyendo la llamada indirecta a través del puntero a función.

### 4.5. Costo del despachador del SysTick (M5)

Restando a la duración total medida en M1 la suma de las tres tareas medidas en M2, M3 y M4 se obtiene el tiempo que insume exclusivamente el bucle despachador de `SysTick_Handler()`:

$$t_{\text{despachador}} = t_{M1} - (t_{M2} + t_{M3} + t_{M4})$$

| Componente                             | Tiempo        | % del total (M1) |
|----------------------------------------|---------------|------------------|
| Refresco del display (M2, brillo máx.) | _____ $\mu$ s | _____ %          |
| FSM del encoder (M3, reposo)           | _____ $\mu$ s | _____ %          |
| Base de tiempo (M4)                    | _____ $\mu$ s | _____ %          |
| Suma de las tres tareas                | _____ $\mu$ s | _____ %          |
| Total medido (M1)                      | _____ $\mu$ s | 100 %            |
| <b>Despachador (diferencia)</b>        | _____ $\mu$ s | _____ %          |

Cuadro 4: Descomposición del tiempo de la ISR del SysTick.

El bucle despachador agrega \_\_\_\_\_  $\mu$ s de sobrecarga sobre las tareas, un \_\_\_\_\_ % del total. Este es el costo concreto de permitir que `App_Init()` registre tareas con `HAL_Scheduler_AddTask()`

sin que el MCAL conozca su existencia. El resultado \_\_\_\_\_ (justifica / no justifica) mantener el mecanismo genérico frente a llamar a las tres funciones en forma directa dentro de la ISR.

#### 4.6. Costo del despachador de puerto (M6)

Comparando las dos filas de M6 en la Tabla ??, el cuerpo de `IRQ_CardClk()` por sí solo consume \_\_\_\_\_  $\mu s$ , mientras que la interrupción completa — con `gpioIRQDispatch()` incluido — consume \_\_\_\_\_  $\mu s$ . La diferencia,

$$\Delta_{\text{PORT}} = \text{_____} \mu s,$$

es el costo de recorrer las 32 posiciones del puerto para encontrar cuál generó la interrupción, aunque sólo una tenga el *flag* activo. Representa un \_\_\_\_\_ % adicional sobre el tiempo de la ISR. El resultado \_\_\_\_\_ (se justifica / no se justifica) frente a la alternativa de que cada driver implemente su propio `PORTB_IRQHandler`, considerando que a cambio cualquier módulo puede registrar un callback por pin sin tocar el MCAL.

El período entre bits medido en M6 (\_\_\_\_\_  $\mu s$ ) corresponde a la velocidad con la que se pasó la tarjeta en el ensayo; se retoma en el análisis de latencia (M8).

#### 4.7. Asimetría entre las dos ISR del lector (M7 vs. M6)

La ISR de `ENABLE` (M7) consume \_\_\_\_\_  $\mu s$  en la apertura y \_\_\_\_\_  $\mu s$  en el cierre, es decir del orden de \_\_\_\_\_ veces el tiempo de la ISR de `CLOCK` medida en M6. La diferencia se explica porque `IRQ_CardEnable()` reconfigura la interrupción de `CLOCK` mediante `gpioIRQ()`, operación que implica lectura-modificación-escritura sobre el registro PCR del puerto, limpieza del *flag* en ISFR y habilitación en el NVIC.

Esta asimetría no representa un problema de diseño: la ISR larga ocurre sólo dos veces por pasada, en momentos en que no hay bits llegando, mientras que la ISR corta es la que se repite decenas de veces. El trabajo pesado está ubicado donde no hay presión de tiempo.

#### 4.8. Validación de la política de prioridades (M8)

Este es el resultado más relevante del informe. La Tabla ?? muestra una latencia máxima de \_\_\_\_\_  $\mu s$  entre el flanco físico de `CLOCK` y la entrada efectiva a `IRQ_CardClk()`. Comparando ese valor contra la duración de la ISR del SysTick medida en M1 (\_\_\_\_\_  $\mu s$ ):

- Si la latencia máxima es notablemente menor a la duración de la ISR del SysTick, queda demostrado que `PORTB` efectivamente interrumpe al SysTick, y la asignación de prioridades (`APP_NVIC_PRIO_SYSTICK = 1`) funciona como se diseñó.
- Si en cambio la latencia máxima se acerca a la duración de la ISR del SysTick, los flancos de `CLOCK` estarían esperando a que el SysTick termine, y el sistema estaría cerca de perder bits en pasadas rápidas.

El resultado obtenido indica que \_\_\_\_\_, lo que \_\_\_\_\_ (valida / invalida) experimentalmente la decisión de prioridades tomada en `App_Init()`.

Relacionando la latencia máxima con el período entre bits medido en M6 (\_\_\_\_\_  $\mu s$ ), el margen del sistema frente al peor caso es de \_\_\_\_\_ veces: la tarjeta podría pasarse esa cantidad de veces más rápido antes de que la latencia de interrupción comprometa la captura de un bit.

#### 4.9. Verificación del contrato de tiempo del encoder (M9)

En reposo, el super-loop completa una vuelta cada \_\_\_\_\_  $\mu s$  (Tabla ??), muy por debajo del límite de 1 ms documentado en `encoder.h`, así que el contrato de `getEncoderEvent()` \_\_\_\_\_ (se cumple / no se cumple).

Durante la decodificación de una tarjeta el período sube a \_\_\_\_\_  $\mu s$ , porque el lazo ejecuta además `getCardData()` (búsqueda del *Start Sentinel* y verificación de paridad de cada carácter). Aun en ese caso el período se mantiene \_\_\_\_\_ (por debajo / por encima) del límite de 1 ms, lo que confirma que separar la captura (en la ISR) de la decodificación (en el lazo principal) es una decisión de diseño correcta: un retardo en la decodificación no causa pérdida de datos, porque los bits ya están guardados en el arreglo.

#### 4.10. Conclusiones

1. El sistema \_\_\_\_\_ (cumple / no cumple) su restricción de tiempo real fundamental: la ISR del SysTick consume \_\_\_\_\_ % de su período de 1 ms.
2. El refresco del display es el componente dominante (\_\_\_\_\_ % del tiempo de interrupción). Una futura optimización de rendimiento debería atacar el *bit-banging* hacia los shift-registers, por ejemplo migrándolo a SPI por hardware.
3. La latencia máxima medida para el lector (\_\_\_\_\_  $\mu s$ ) frente a la duración de la ISR del SysTick (\_\_\_\_\_  $\mu s$ ) \_\_\_\_\_ (confirma / refuta) que bajar la prioridad del SysTick fue la decisión correcta.
4. Los mecanismos genéricos de despacho (SysTick y GPIO) suman \_\_\_\_\_  $\mu s$  de sobrecarga total, un \_\_\_\_\_ % del presupuesto de CPU. El equipo considera que este costo \_\_\_\_\_ (se justifica / no se justifica) frente a la ganancia en modularidad y portabilidad de la arquitectura en capas.
5. Todas las mediciones corresponden a -00. Se estima que con -02 los tiempos absolutos se reducirían \_\_\_\_\_, aunque las proporciones relativas entre rutinas se mantendrían.

## A. Capturas de osciloscopio

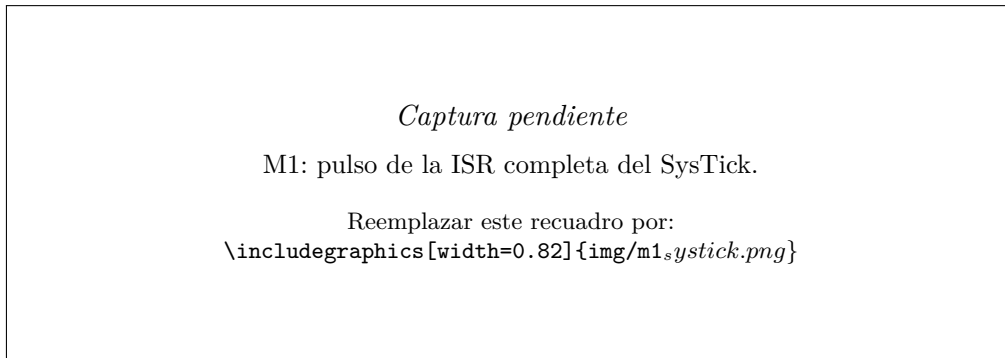


Figura 1: M1: pulso de la ISR completa del SysTick.

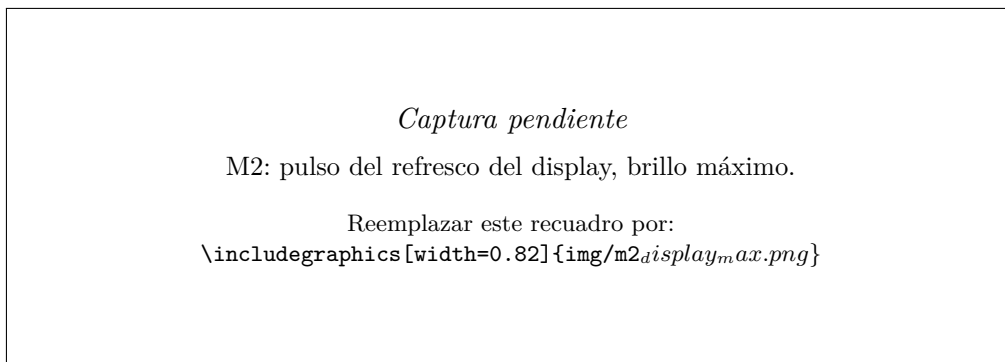


Figura 2: M2: pulso del refresco del display, brillo máximo.

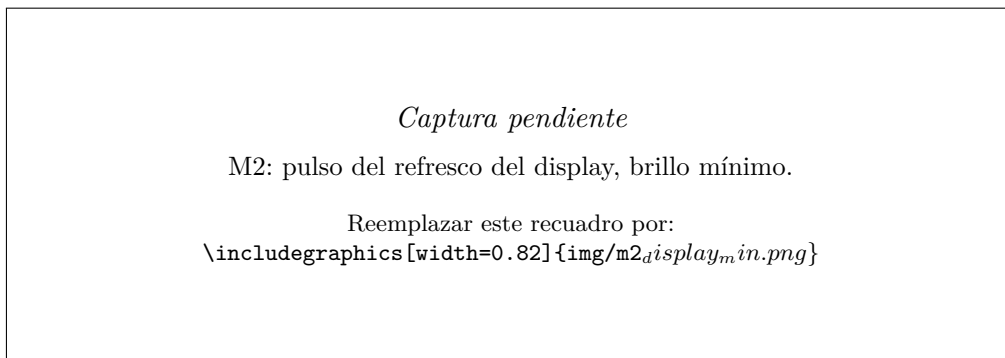


Figura 3: M2: pulso del refresco del display, brillo mínimo.

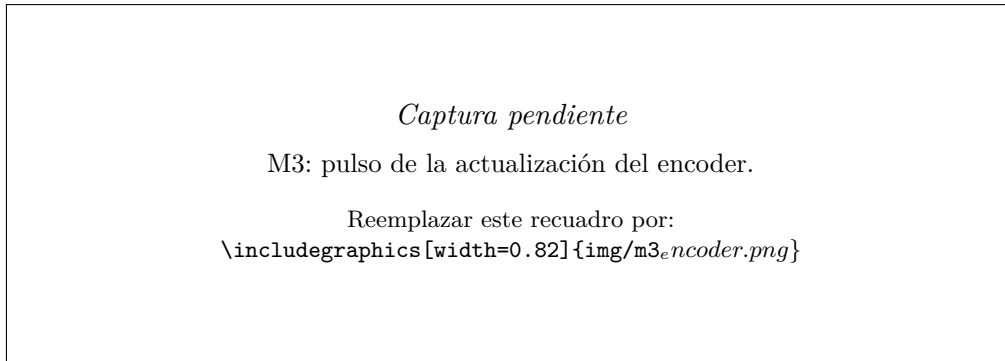


Figura 4: M3: pulso de la actualización del encoder.

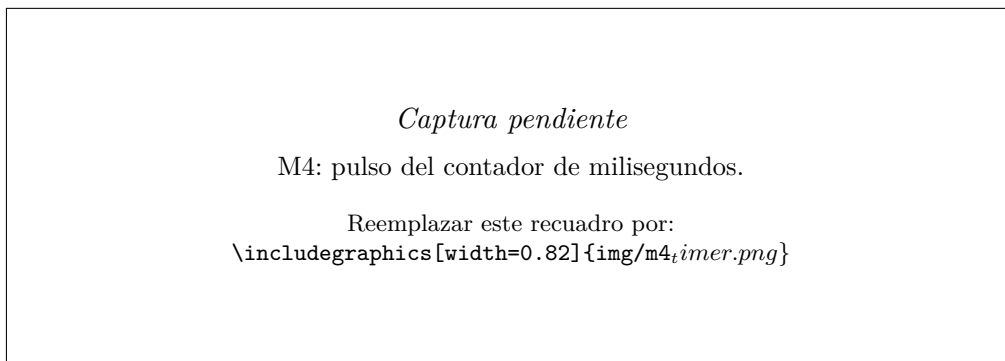


Figura 5: M4: pulso del contador de milisegundos.

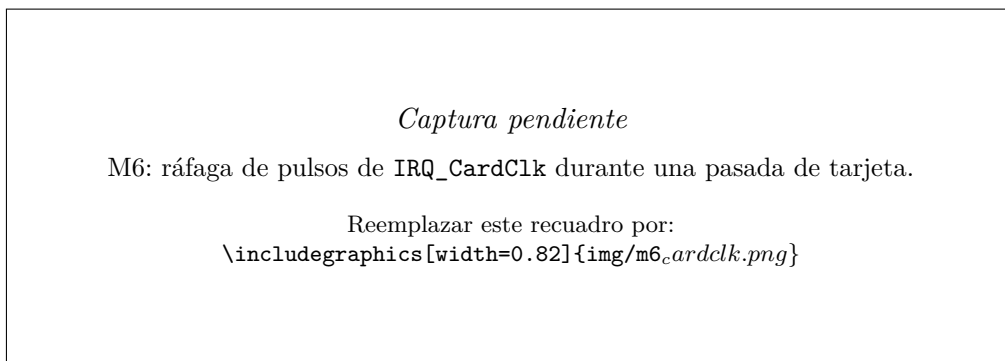


Figura 6: M6: ráfaga de pulsos de IRQ\_CardClk durante una pasada de tarjeta.

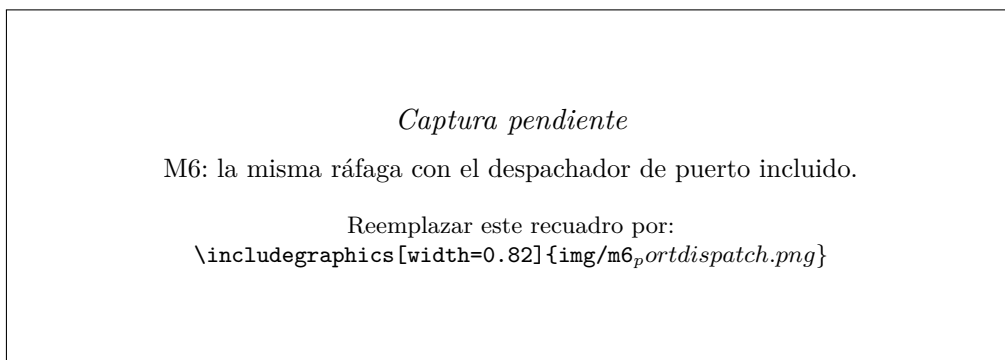


Figura 7: M6: la misma ráfaga con el despachador de puerto incluido.

*Captura pendiente*

M7: pulsos de apertura y cierre de IRQ\_CardEnable.

Reemplazar este recuadro por:  
`\includegraphics[width=0.82]{img/m7_cardenable.png}`

Figura 8: M7: pulsos de apertura y cierre de IRQ\_CardEnable.

*Captura pendiente*

M8: flanco de CLOCK (CH1) contra entrada a la ISR (CH2).

Reemplazar este recuadro por:  
`\includegraphics[width=0.82]{img/m8_atencia.png}`

Figura 9: M8: flanco de CLOCK (CH1) contra entrada a la ISR (CH2).

*Captura pendiente*

M9: período del super-loop en reposo.

Reemplazar este recuadro por:  
`\includegraphics[width=0.82]{img/m9_ooprepos.png}`

Figura 10: M9: período del super-loop en reposo.

*Captura pendiente*

M9: período del super-loop decodificando una tarjeta.

Reemplazar este recuadro por:  
`\includegraphics[width=0.82]{img/m9_ooptarjeta.png}`

Figura 11: M9: período del super-loop decodificando una tarjeta.